LAB PROGRAM DAY 2

1.Program to Find the First Palindromic String in an Array

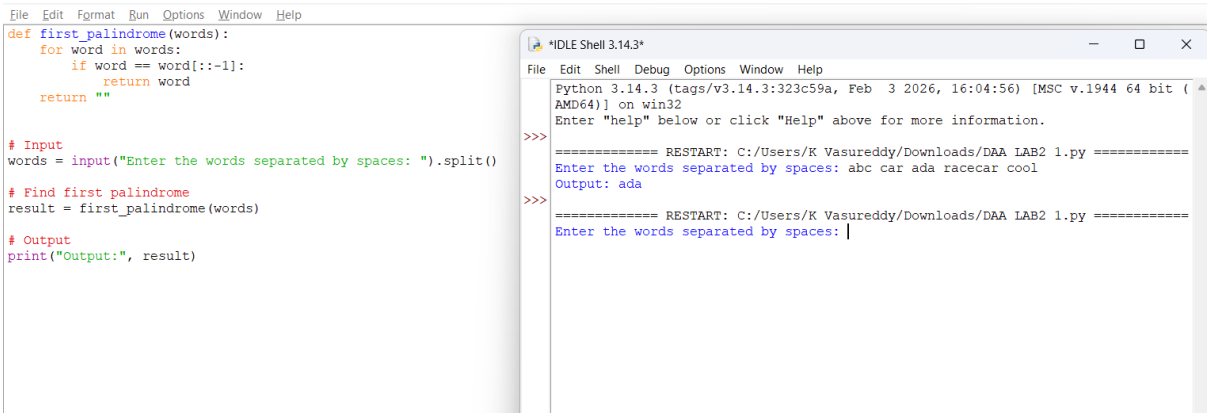
Aim

To write a Python program to find the first palindromic string in an array of strings. If no palindrome exists, return an empty string "".

Procedure

1. Start the program.
2. Read the list of strings.
3. Traverse each string from left to right.
4. Reverse the current string using slicing [::-1].
5. Compare the string with its reverse.
6. If they are equal, return that string immediately.
7. If no palindrome is found, return "".
8. Display the result.
9. Stop.

Program&Output:



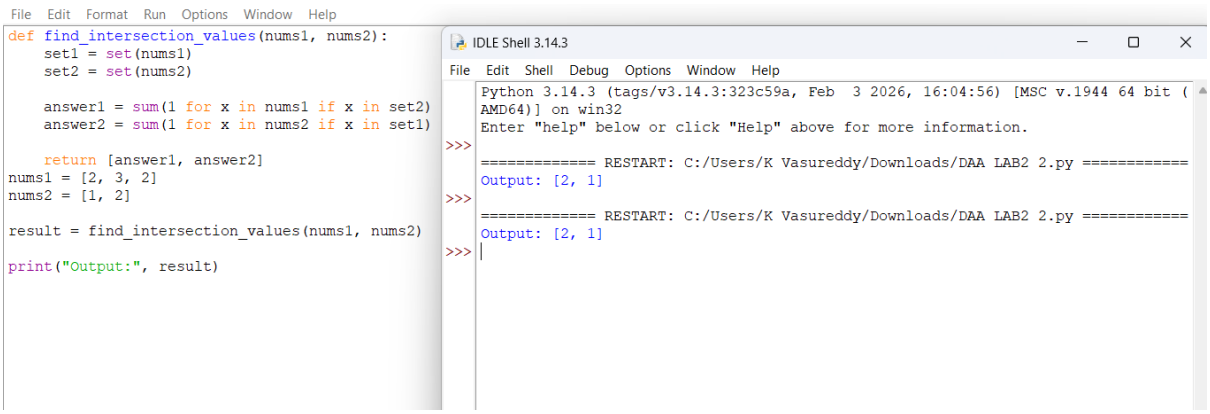
2. Program to Find the Number of Common Elements in Two Arrays

Aim: To write a Python program to calculate the number of elements in nums1 that exist in nums2 and the number of elements in nums2 that exist in nums1.

Procedure

1. Start the program.
2. Read two integer arrays nums1 and nums2.
3. Create a set from nums2 for quick searching.
4. Count the elements of nums1 that are present in nums2.
5. Create a set from nums1.
6. Count the elements of nums2 that are present in nums1.
7. Store the two counts as [answer1, answer2].
8. Display the result.
9. Stop the program.

Program&Output:



3. Sum of Squares of Distinct Counts of All Subarrays

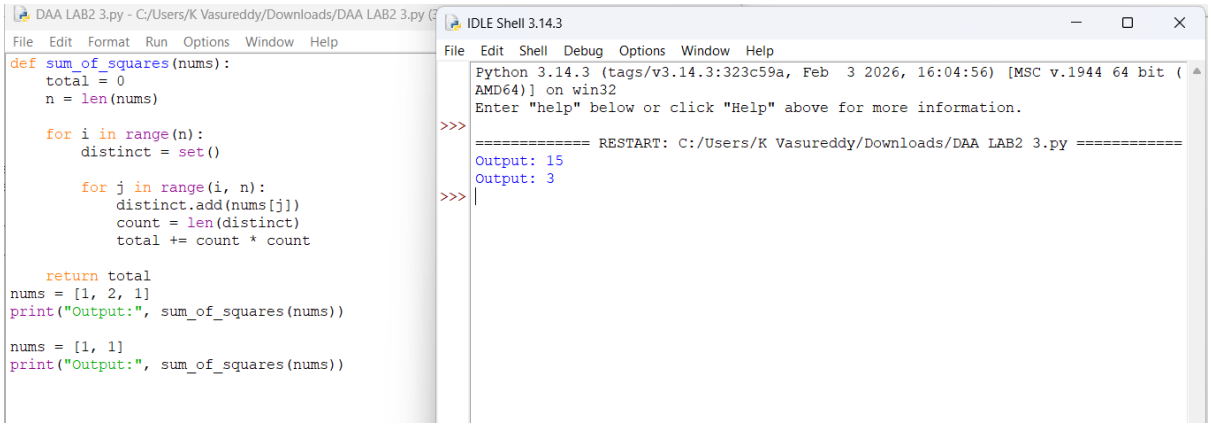
Aim

To write a Python program to find the sum of the squares of the number of distinct elements in every possible subarray.

Procedure

1. Start the program.
2. Initialize total = 0.
3. Select each possible starting index.
4. Maintain a set of distinct elements while extending the subarray.
5. For every subarray, find the number of distinct elements.
6. Add the square of the distinct count to total.
7. Return and display total.
8. Stop the program.

Program&Output:



4. Count Pairs Satisfying the Given Conditions

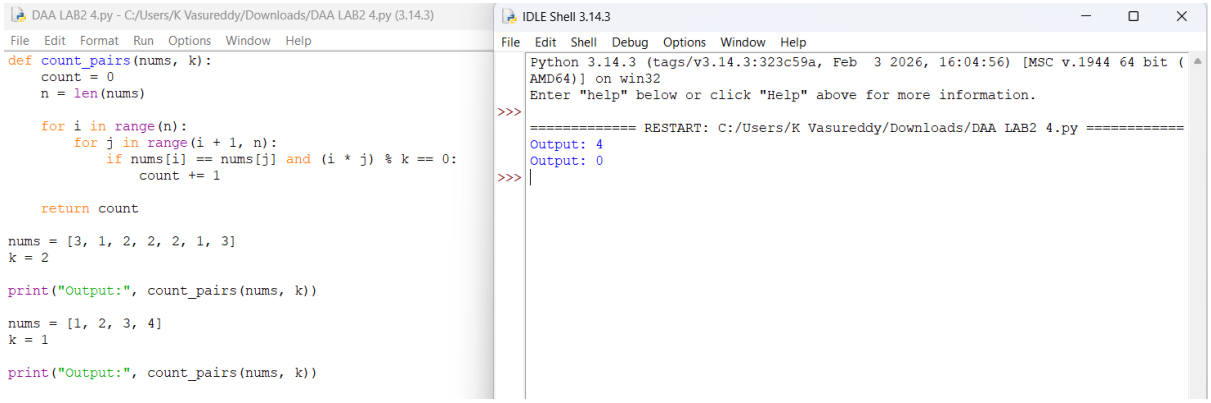
Aim

To write a Python program to count pairs (i, j) where nums[i] == nums[j] and (i * j) is divisible by k.

Procedure

1. Start the program.
2. Read the array nums and integer k.
3. Consider every pair of indices (i, j) where i < j.
4. Check whether nums[i] == nums[j].
5. Check whether (i * j) % k == 0.
6. If both conditions are true, increment the count.
7. Return the total number of valid pairs.
8. Stop the program.

Program&Output



5. Find the Maximum Value in an Array

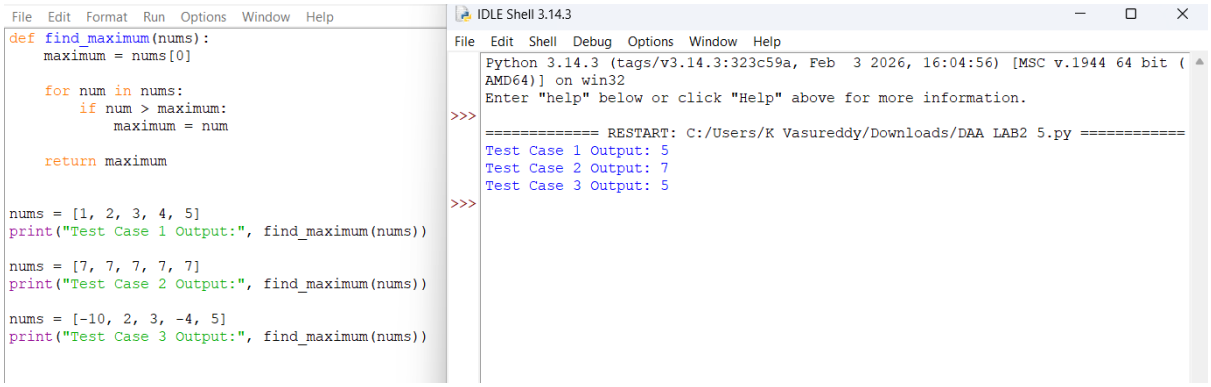
Aim

To write an efficient Python program to find the maximum element in an integer array with minimum time complexity.

Procedure

1. Start the program.
2. Read the array of integers.
3. Assume the first element is the maximum.
4. Traverse the array once.
5. Compare each element with the current maximum.
6. Update the maximum whenever a larger element is found.
7. Display the maximum value.
8. Stop the program

Program & Output



```
File  Edit  Format  Run  Options  Window  Help
def find_maximum(nums):
    maximum = nums[0]

    for num in nums:
        if num > maximum:
            maximum = num

    return maximum


nums = [1, 2, 3, 4, 5]
print("Test Case 1 Output:", find_maximum(nums))

nums = [7, 7, 7, 7, 7]
print("Test Case 2 Output:", find_maximum(nums))

nums = [-10, 2, 3, -4, 5]
print("Test Case 3 Output:", find_maximum(nums))
```

```
IDLE Shell 3.14.3
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/K Vasureddy/Downloads/DAA LAB2 5.py ===========
Test Case 1 Output: 5
Test Case 2 Output: 7
Test Case 3 Output: 5
>>>
```

6. Sort the List and Find the Maximum Element

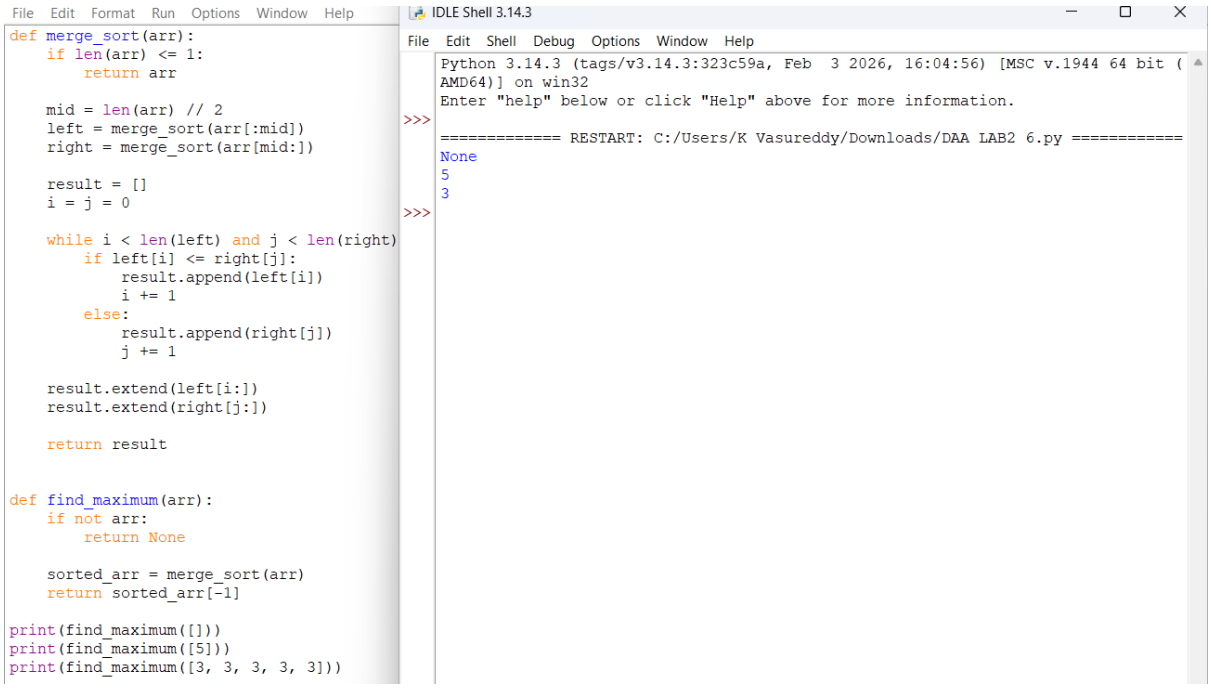
Aim

To write a Python program that first sorts a list using an efficient sorting algorithm and then finds the maximum element from the sorted list.

Procedure

1. Start the program.
2. Read the list of numbers.
3. Check whether the list is empty.
4. If empty, display an appropriate message.
5. Sort the list using Merge Sort.
6. The last element of the sorted list is the maximum.
7. Display the maximum element.
8. Stop the program

Program & Output



```
File  Edit  Format  Run  Options  Window  Help
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


def find_maximum(arr):
    if not arr:
        return None

    sorted_arr = merge_sort(arr)
    return sorted_arr[-1]

print(find_maximum([]))
print(find_maximum([5]))
print(find_maximum([3, 3, 3, 3, 3]))
```

```
IDLE Shell 3.14.3
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============= RESTART: C:/Users/K Vasureddy/Downloads/DAA LAB2 6.py ============
None
5
3
>>>
```

7. Find Unique Elements in a List

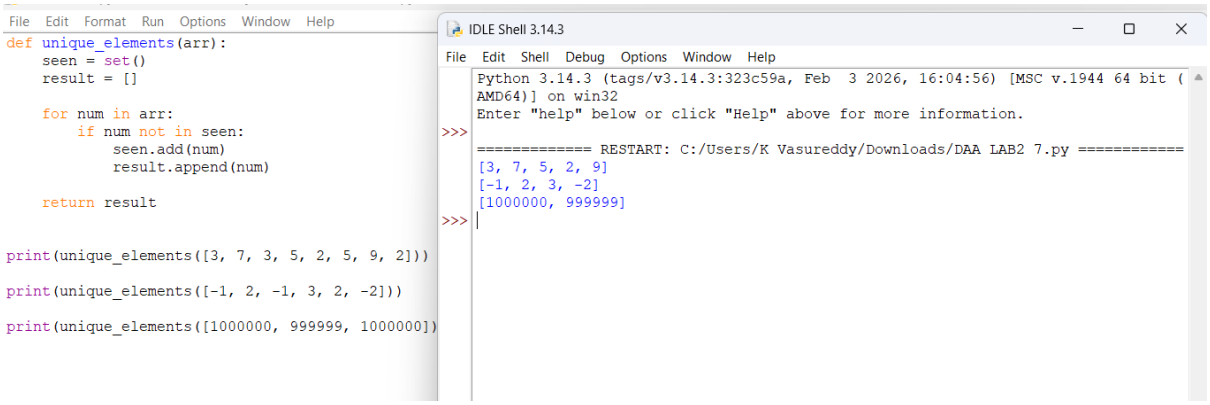
Aim

To write a Python program that creates a new list containing only the unique elements from the given list.

Procedure

1. Start the program.
2. Read the input list.
3. Create an empty set to store elements already seen.
4. Traverse the list.
5. If an element is not present in the set, add it to the set and result list.
6. Display the result list.
7. Stop the program

Program & Output



8. Bubble Sort

Aim

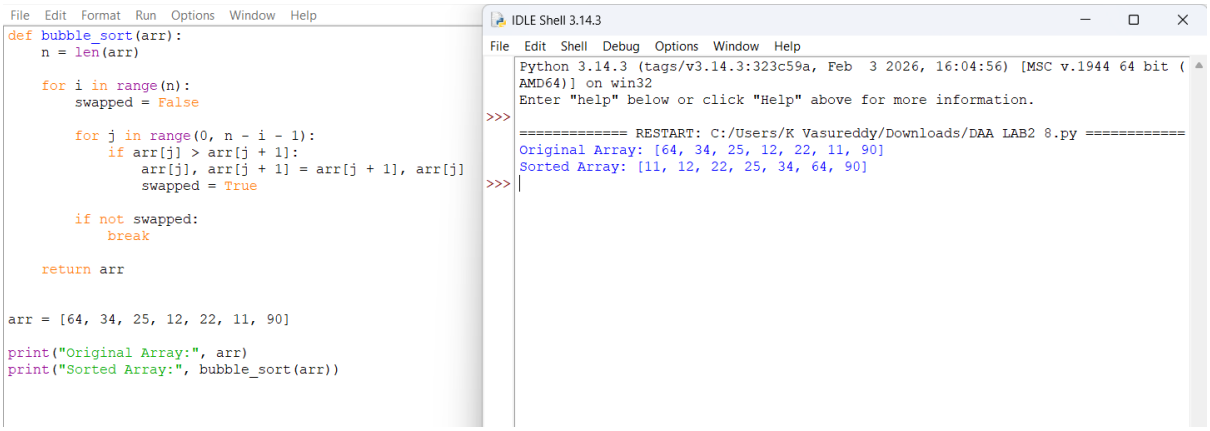
To write a Python program to sort an array of integers in ascending order using the Bubble Sort technique and analyze its time complexity.

Procedure

1. Start the program.
2. Read the array.
3. Compare adjacent elements.
4. If the first element is greater than the second, swap them.
5. Repeat the process for all elements.

6. Continue until the array is sorted.
7. Display the sorted array.
8. Stop the program.

Program & Output



```
File  Edit  Format  Run  Options  Window  Help
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr


arr = [64, 34, 25, 12, 22, 11, 90]

print("Original Array:", arr)
print("Sorted Array:", bubble_sort(arr))
```

```
IDLE Shell 3.14.3
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/K Vasureddy/Downloads/DAA LAB2 8.py ==========
Original Array: [64, 34, 25, 12, 22, 11, 90]
Sorted Array: [11, 12, 22, 25, 34, 64, 90]
>>>
```

9. Binary Search

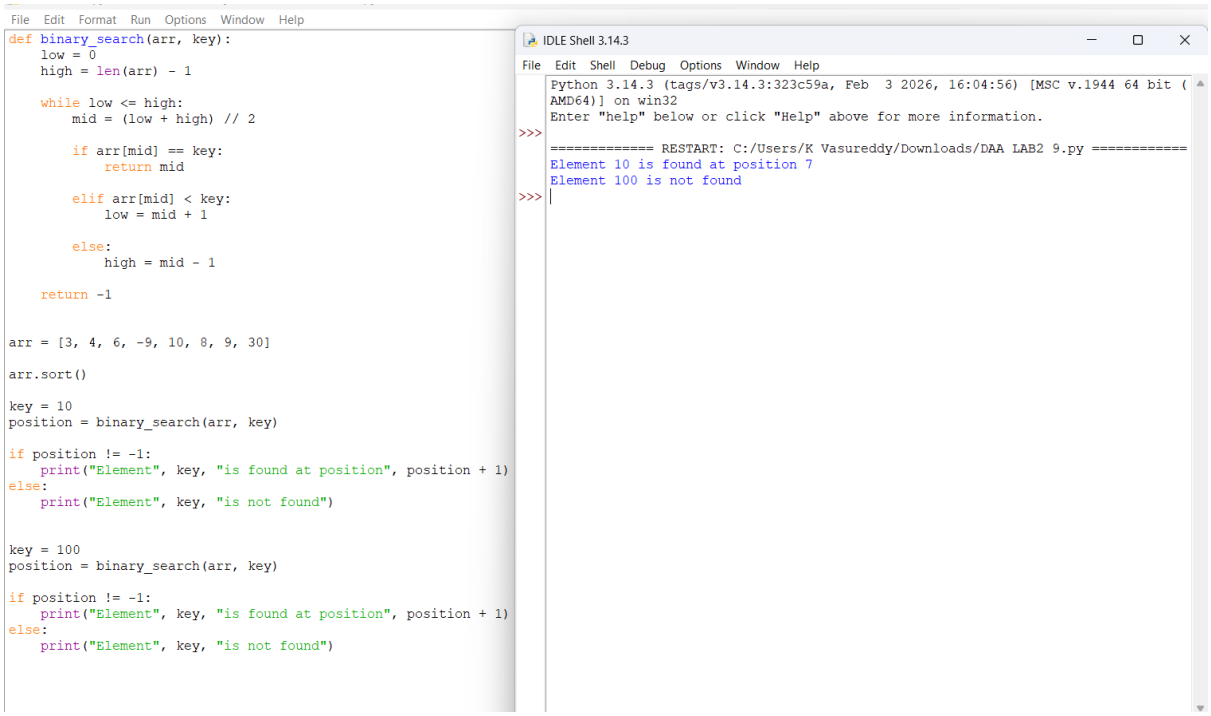
Aim

To write a Python program to search for a given element in a sorted array using Binary Search.

Procedure

1. Start the program.
2. Initialize low = 0 and high = n - 1.
3. Find the middle element.
4. Compare the middle element with the key.
5. If equal, return its position.
6. If the key is greater, search the right half.
7. If the key is smaller, search the left half.
8. Repeat until the element is found or the search range becomes empty.
9. Display the result.
10. Stop the program.

Program & Output



```
File  Edit  Format  Run  Options  Window  Help
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1


arr = [3, 4, 6, -9, 10, 8, 9, 30]

arr.sort()

key = 10
position = binary_search(arr, key)

if position != -1:
    print("Element", key, "is found at position", position + 1)
else:
    print("Element", key, "is not found")


key = 100
position = binary_search(arr, key)

if position != -1:
    print("Element", key, "is found at position", position + 1)
else:
    print("Element", key, "is not found")
```

```
IDLE Shell 3.14.3
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/K Vasureddy/Downloads/DAA LAB2 9.py ==========
Element 10 is found at position 7
Element 100 is not found
>>>
```

10. Sort an Array in Ascending Order in O(n log n)

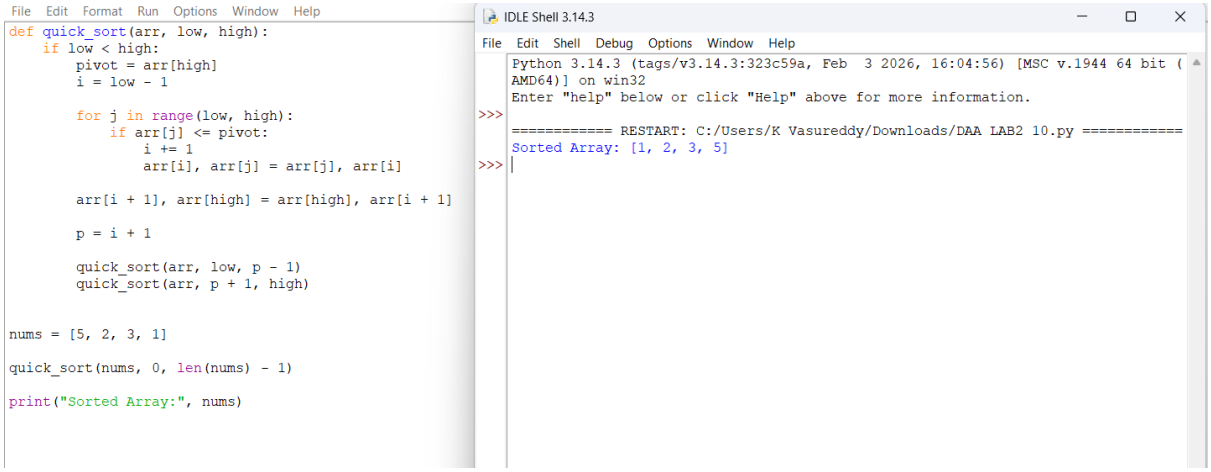
Aim

To write a Python program to sort an array of integers in ascending order without using built-in sorting functions, with O(n log n) time complexity and minimum extra space.

Procedure

1. Start the program.
2. Select the middle element as the pivot.
3. Partition the array around the pivot.
4. Elements smaller than the pivot are placed on the left.
5. Elements greater than the pivot are placed on the right.
6. Recursively sort both partitions.
7. Continue until the array is sorted.
8. Display the sorted array.
9. Stop the program.

Program & Output



File Edit Format Run Options Window Help

```
def quick_sort(arr, low, high):
    if low < high:
        pivot = arr[high]
        i = low - 1

        for j in range(low, high):
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]

        arr[i + 1], arr[high] = arr[high], arr[i + 1]

        p = i + 1

        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)


nums = [5, 2, 3, 1]

quick_sort(nums, 0, len(nums) - 1)

print("Sorted Array:", nums)
```